

Dhirubhai Ambani University
Technology

Formerly DA-IICT

Project

Socket Programming

Lab

G4

Course

IT305 - Computer Networks

Lab Assignment-6

Fault Tolerant Topic-based File Distribution Service

Group: Group 071 + Group 072
Group members:

1. Jay Vaghela · 202401237
2. Dharmraj Vaghela · 202401236
3. Vivek Dodiya · 202401253
4. Atharv Trivedi · 202401230
5. Shlok Topaliya · 202401229
6. Yug Malaviya · 202401256

Table of Contents

1. Project Overview
2. Requirements
3. Design Principles and Assumptions
4. High-Level Architecture
5. Server Data Structures
6. TCP Client-Server Protocol
7. Part I Design
8. Client Design
9. File Transfer Design
10. Part II Fault Injection
11. Part II Case 1: No Session Management
12. Part II Case 2: Session Management and Resume
13. Part II Case 2 Enhanced
14. Performance Measurement Plan
15. Error Handling and Edge Cases
16. Concurrency and Synchronization
17. Module and File Structure
18. Planned Key Functions
19. Build and Execution Design
20. Design Alternatives and Decisions
21. Robustness and Safety Considerations
22. Logging and Additional Features
23. End-to-End Workflows
24. Verification and Automated-Test Compatibility
25. Requirements Traceability Matrix
26. Expected Design Outcome

1. Project Overview

1.1 Problem statement

The system is a TCP-based topic file distribution service. A server receives a root directory from the command line. Each immediate subdirectory below that root is a topic, and the files inside that subdirectory belong to the topic. A client requests one topic and receives every regular file in that topic directory.

The project has two linked parts. Part I establishes the basic service and compares a sequential server with a multi-threaded server. Part II extends the multi-threaded server so that a probabilistically interrupted transfer can either restart from the beginning or resume from a saved checkpoint. Case 2 Enhanced explores a performance improvement while preserving the required command-line interface.

1.2 Motivation and objective

A file transfer can fail after some data has already crossed the network. Restarting is simple but may transmit the same bytes repeatedly. Session management records enough progress to avoid unnecessary retransmission. The objective is therefore to design a small, understandable POSIX socket system that demonstrates TCP framing, file I/O, concurrency, controlled fault injection, recovery, and measurable performance.

The design deliberately avoids HTTP, FTP, WebSockets, REST, remote procedure call frameworks, and external libraries. The application protocol is defined directly over TCP so that the behavior of byte-stream framing, partial `send()` / `recv()`, and recovery is explicit.

1.3 Scope

The design covers the four implementation directories required by the assignment: `Part1/`, `Part2_Case1/`, `Part2_Case2/`, and `Part2_Case2_Enhanced/`. Each directory is planned to contain a `server`, a `client`, and a `Makefile`.

The design does not include final source code, measured performance values, fabricated graphs, or claims about completed experiments. The dataset path is not embedded in the design or implementation; it is supplied through `<topics_root_dir>`.

1.4 Relationship between the parts

Part I provides the common socket, topic lookup, framing, file-transfer, and client logic. Its single-threaded server handles one accepted connection at a time. Its multi-threaded server keeps the same protocol but creates one worker thread per client.

Part II reuses the multi-threaded architecture. Case 1 adds server-side fault injection but deliberately retains no progress state after a connection fails. Case 2 adds a session identifier and checkpoint exchange. Case 2 Enhanced builds on Case 2 and divides a topic into independent file or byte-range work units over multiple TCP connections. The enhanced design has a conservative default so that the mandatory client invocation remains valid.

2. Requirements

2.1 Functional requirements

ID	Requirement	Description	Part
FR-01	TCP service	Use TCP sockets for client–server communication.	I, II
FR-02	Topic lookup	Treat each directory immediately under the supplied root as a topic.	I, II
FR-03	Topic request	Accept a request equivalent to `GET <topic>`.	I, II
FR-04	File distribution	Send all regular files belonging to the requested topic.	I, II
FR-05	Sequential server	Serve clients sequentially using one request-processing flow.	I
FR-06	Concurrent server	Serve simultaneous clients with one worker thread per client.	I, II
FR-07	Fault injection	Interrupt active transfers according to the configured failure probability.	II
FR-08	Case 1 restart	After a failure, discard progress and restart the entire topic.	II Case 1
FR-09	Case 2 resume	Preserve and validate a checkpoint, then resume from it.	II Case 2
FR-10	Enhanced transfer	Add a documented performance enhancement to session-managed transfer.	II Enhanced
FR-11	Output layout	Write received files under `<output_dir>/<topic>/`.	I, II
FR-12	Required server CLI	Part I: `./server <port> <topics_root_dir>`; Part II: `./server <port> <topics_root_dir> <failure probability>`.	I, II

ID	Requirement	Description	Part
FR-13	Required client CLI	`./client <server_ip> <server_port> <topic> <output_dir>`.	I, II

2.2 Non-functional and design requirements

ID	Requirement	Design response
NFR-01	Binary-safe protocol	Control frames have explicit lengths; file payloads are read by an explicit byte count.
NFR-02	Partial I/O correctness	All fixed-size headers and payloads use `send_all()` / `recv_all()`-style loops.
NFR-03	No hard-coded environment values	Port, IP, root directory, topic, and output directory come from arguments.
NFR-04	Automated-test compatibility	Exact executable names, directory names, and mandatory argument forms are retained.
NFR-05	Implementation feasibility	Blocking sockets and thread-per-client are the baseline; more complex mechanisms are isolated to the enhanced case.
NFR-06	Recoverability	A disconnect is detected, logged, and handled without corrupting completed output files.
NFR-07	Measurability	Monotonic timestamps and byte counters define reproducible planned metrics.
NFR-08	Safe path handling	Topic and filename components are validated before filesystem access.
NFR-09	Academic integrity	All decisions are proposed for this project; no external implementation is copied or represented as tested.

3. Design Principles and Assumptions

The server treats the supplied root as authoritative for topic discovery. A topic is the name of an immediate child directory. The implementation will enumerate directories at startup or on

demand, but it will not assume any particular absolute path or animal-name spelling.

Only regular files are transferred. Symbolic links, nested directories, and special files are excluded unless the implementation later documents a deliberate extension. Filenames are treated as opaque byte strings after validation; the protocol carries their explicit length rather than relying on newline delimiters.

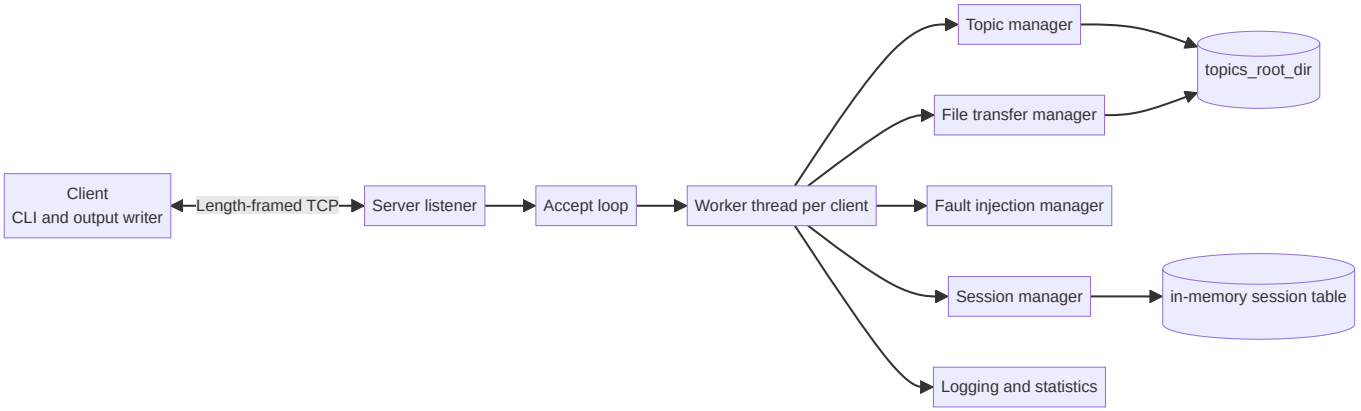
The baseline protocol sends files sequentially. This is sufficient for the required service and makes checkpoint semantics unambiguous: a checkpoint identifies the current file and the number of bytes committed for that file, plus the files already completed in deterministic directory order.

A design decision is made to use **blocking TCP sockets** for Part I, Case 1, and Case 2. Blocking sockets reduce the number of event states that must be implemented in an undergraduate laboratory project. The enhanced case may use several blocking connections rather than non-blocking I/O; this provides a clear enhancement without changing the baseline worker model.

---

4. High-Level Architecture

4.1 Overall architecture



The **client** parses the required arguments, opens a connection, requests a topic, creates the output directory, receives metadata and bytes, and records local progress. The **listener** owns the listening socket and accepts clients. A **worker** owns one connected socket and one client transaction.

The **topic manager** maps a validated topic name to a directory and produces a stable ordered file manifest. The **file transfer manager** opens files, seeks to a validated offset, and sends metadata and exact payload lengths. The **session manager** is enabled only where required and

protects the shared session table. The **fault injection manager** makes and logs eligible failure decisions. Logging and statistics provide diagnostics and planned performance counters.

4.2 Architecture by part

Version	Main control flow	Shared state
Part I single-threaded	`accept()` → handle request and files → close → next `accept()`	Read-only topic root; process-local counters
Part I multi-threaded	`accept()` → create detached worker → immediately accept again	Read-only topic data; mutex-protected counters/logging if shared
Part II Case 1	Multi-threaded flow plus failure decision; reconnect restarts	Failure configuration and counters
Part II Case 2	Multi-threaded flow plus session creation, checkpoint validation, and resume	Mutex-protected session table and logs
Case 2 Enhanced	Coordinator uses several connections for independent work units	Session/work-unit table, counters, and failure state

4.3 Component contracts

Component	Inputs	Outputs	State
Argument/configuration layer	`argc`, `argv`	Validated runtime configuration	Port, root path, probability, optional enhancement defaults
Topic manager	Topic string, root path	Manifest or typed error	Topic-to-directory mapping and ordered file metadata
Protocol layer	Socket and typed messages	Parsed frames and serialized frames	Frame buffers and protocol limits
File transfer manager	Manifest, file, offset, socket	File metadata, payload, completion	Current descriptor, offset, bytes sent
Session manager	Session ID, topic, checkpoint	Validated resume plan	Session record, status, expiry timestamp
Fault manager	Probability, eligible event	Continue or interrupt decision	Seed/configuration and decision count
Worker	Connected socket	Complete, error, or interrupted transaction	Client address, request state, local counters
Statistics/logger	Events and counters	Human-readable/CSV-compatible records	Mutex-protected output stream and aggregate counters

5. Server Data Structures

The following are conceptual structures. The final C representation may use structs, bounded strings, dynamic arrays, or linked lists, but the fields and invariants remain the same.

5.1 Configuration

```
ServerConfig:
    listen_port
    topics_root_dir
    failure_probability       // Part II only, in [0.0, 1.0]
    fault_seed_policy         // default random seed; optional reproducib
    max_clients
    session_ttl_seconds
    chunk_size
```

`listen_port` and `topics_root_dir` are mandatory arguments. `failure_probability` is mandatory for Part II. The remaining values are implementation constants or optional configuration with documented defaults; they are not additional mandatory CLI arguments.

5.2 Topic map and manifest

```
TopicEntry:
    topic_name
    directory_path

FileEntry:
    filename
    relative_path
    file_size
    regular_file

TopicManifest:
    topic_name
    directory_path
    files[]
    manifest_version_or_snapshot_time
```

The conceptual mapping is `topic_name → directory_path`. The root path is canonicalized once, while a requested topic is accepted only if it is a single valid directory component. The manifest is sorted lexicographically by filename to make client and server checkpoints comparable across reconnects. The final implementation will use actual file metadata obtained from `stat()` when building the manifest.

5.3 Client/request state

```
RequestState:
    client_address
    requested_topic
    manifest
    current_file_index
    current_file_offset
    bytes_sent_on_connection
    bytes_useful_for_transfer
    status
```

Each worker owns its request state, so these fields do not require a global mutex. The manifest itself is read-only after construction. If manifests are cached, replacement of a cache entry is synchronized.

5.4 Session/checkpoint state

```
Session:
    session_id
    topic
    manifest_fingerprint
    current_file_index
    current_file_name
    current_file_offset
    completed_files
    status                  // ACTIVE, INTERRUPTED, COMPLETE, EXPIRED
    last_checkpoint_time
    last_activity_time
    client_identity_hint
    bytes_useful
    bytes_redundant
```

`session_id` identifies the logical transfer across TCP connections. `topic` prevents a session from being reused for a different request. `manifest_fingerprint` detects a changed topic directory. The file index and name provide consistency checks. `current_file_offset` is the number of bytes that the protocol has committed as received by the client.

`completed_files` records files before the current file. Timestamps support expiry and diagnostics. Status prevents an expired or completed session from being resumed incorrectly.

The server is authoritative for the committed checkpoint. The client keeps a local copy to reconnect quickly, but a client-provided checkpoint is a proposal, not proof. The server returns a validated resume point. This choice prevents a client from claiming that bytes were received when the server has not recorded them. The implementation must make the checkpoint update operation atomic with respect to the session mutex.

5.5 Statistics

```
TransferStats:
    session_id_or_connection_id
    topic
    total_socket_bytes_sent
    total_socket_bytes_received
    useful_file_bytes_delivered
    redundant_file_bytes_delivered
    start_monotonic_ns
    end_monotonic_ns
    failure_count
    reconnect_count
    completion_status
```

Protocol headers are counted in `total_socket_bytes_sent` if the counter is intended to represent socket traffic. File payload bytes are counted separately as useful or redundant bytes. The report will state which counter is used for each graph so that overhead calculations are not ambiguous.

5.6 Synchronization requirements

The topic map and file data are read-only during a transfer and need no per-read lock. The session table, shared statistics, and log output need mutex protection. A worker must not hold the session mutex while doing disk or network I/O. It should copy the checkpoint or update a small record under the mutex, release the lock, and then perform the expensive operation.

---

6. TCP Client–Server Protocol

6.1 Framing decision

TCP provides an ordered byte stream, not message boundaries. A newline-only protocol is unsuitable for binary payloads because file bytes may contain any value. The selected protocol uses a fixed-size binary frame header followed by a payload:

```
FrameHeader {
    magic       2 bytes     // fixed protocol marker, e.g. 0x5446
    version     1 byte
    type        1 byte
    flags       2 bytes
    reserved    2 bytes
    payload_len 8 bytes     // unsigned 64-bit network byte order
}
Payload: payload_len bytes
```

All multi-byte integers are encoded in network byte order. The receiver first calls `recv_all()` for the fixed header, validates the magic, version, type, and maximum payload length, and then calls `recv_all()` for exactly `payload_len` bytes. Control payloads are UTF-8 or fixed binary fields as documented below. File data is never parsed as a control payload.

A file payload is split into `FILE_DATA` frames. Each frame has a bounded payload, such as the configured chunk size, and the metadata gives the total file size and starting offset. The receiver knows that the next `file_size - start_offset` payload bytes belong to that file because the server sends a declared length and a `FILE_END` frame. It does not search for a delimiter inside binary data.

6.2 Common message types

Type	Sender → receiver	Payload fields	Purpose
`GET`	Client → server	`topic_len`, `topic`	Request a topic in Part I and Case 1.
`TOPIC_OK`	Server → client	`session_id?`, `file_count`, manifest entries or manifest fingerprint	Accept the topic and describe the transfer.
`TOPIC_ERROR`	Server → client	error code, message length, message	Reject an invalid or unavailable topic.
`FILE_BEGIN`	Server → client	file index, filename length/name, total size, start offset	Begin or resume one file.
`FILE_DATA`	Server → client	raw file bytes	Carry one chunk of binary data.
`FILE_END`	Server → client	file index, final offset, file size	Confirm the file boundary and completion.
`TRANSFER_END`	Server → client	status, file count, useful bytes	Confirm topic completion.
`ERROR`	Either direction	error code and diagnostic text	Report a recoverable or terminal protocol error.
`CLOSE`	Either direction	reason code	Graceful close when no transfer remains.

The human-readable notation `GET <topic>` describes the logical request required by the assignment. On the wire, the request is a length-framed `GET` frame so that topic bytes and control boundaries are unambiguous.

6.3 Part I and Case 1 message sequence

Step	Sender	Frame	Expected behavior
1	Client	`GET(topic)`	Request one topic.
2	Server	`TOPIC_OK(file_count, manifest)` or `TOPIC_ERROR`	Validate topic and advertise files.
3	Server	Repeated `FILE_BEGIN`, `FILE_DATA`, `FILE_END`	Send each file from offset zero.
4	Server	`TRANSFER_END`	Mark this connection's transfer complete.
5	Client	Optional `CLOSE`	Close cleanly after verification.

In Case 1, a failure can occur after any eligible data-transfer decision. The client reconnects and sends the same `GET`; the server creates a new ordinary transfer from file zero. No session ID or checkpoint is sent.

6.4 Case 2 session messages

Step	Sender	Frame	Payload fields
1	Client	`SESSION_START`	topic, client nonce, requested mode
2	Server	`SESSION_ACCEPT`	session ID, manifest fingerprint, file count, checkpoint zero
3	Client	`CHECKPOINT_REPORT`	session ID, local file index/name, local offset, completed-file list or compact prefix
4	Server	`RESUME_ACCEPT`	session ID, authoritative file index/name, authoritative offset, completed prefix
5	Server	File frames	Start at the validated point.
6	Either	`TRANSFER_INTERRUPTED` or connection close	Failure may make the client reconnect.
7	Client	`SESSION_RESUME`	session ID, topic, last local checkpoint
8	Server	`RESUME_ACCEPT` or `RESUME_ERROR`	Validate session and return authoritative point.
9	Server	Remaining file frames and `TRANSFER_END`	Complete the session.

`SESSION_START` is used on the first connection in Case 2. If the initial connection fails before the client receives a session ID, the client starts a new session. If it receives a session ID, it uses `SESSION_RESUME` after reconnecting.

6.5 Case 2 Enhanced messages

The enhanced case uses the same session identity but adds work-unit allocation:

```
WORK_PLAN(session_id, connection_count, units[])
WORK_ASSIGN(session_id, unit_id, file_index, start_offset, end_offset)
WORK_DATA(unit_id, sequence_number, bytes)
WORK_END(unit_id, checksum_optional, final_offset)
SESSION_PROGRESS(session_id, unit_id, committed_offset)
```

The baseline default is one connection, which is behaviorally equivalent to Case 2. When the client requests more connections through an optional trailing argument, the server assigns independent files first. If there are fewer files than connections, it may split large files into non-overlapping byte ranges aligned to the configured chunk size. The client writes each range at the specified offset and marks the unit complete only after all bytes in that unit have been received.

6.6 Protocol limits and error codes

The implementation will enforce maximum topic and filename lengths, a maximum control-frame payload, and a maximum file-data chunk. Proposed error codes include `BAD_VERSION`, `MALFORMED_FRAME`, `INVALID_TOPIC`, `TOPIC_NOT_FOUND`, `FILE_ACCESS`, `SESSION_NOT_FOUND`, `SESSION_EXPIRED`, `CHECKPOINT_CONFLICT`, `OFFSET_INVALID`, `SESSION_COMPLETE`, `DISK_WRITE`, and `INTERNAL_ERROR`.

---

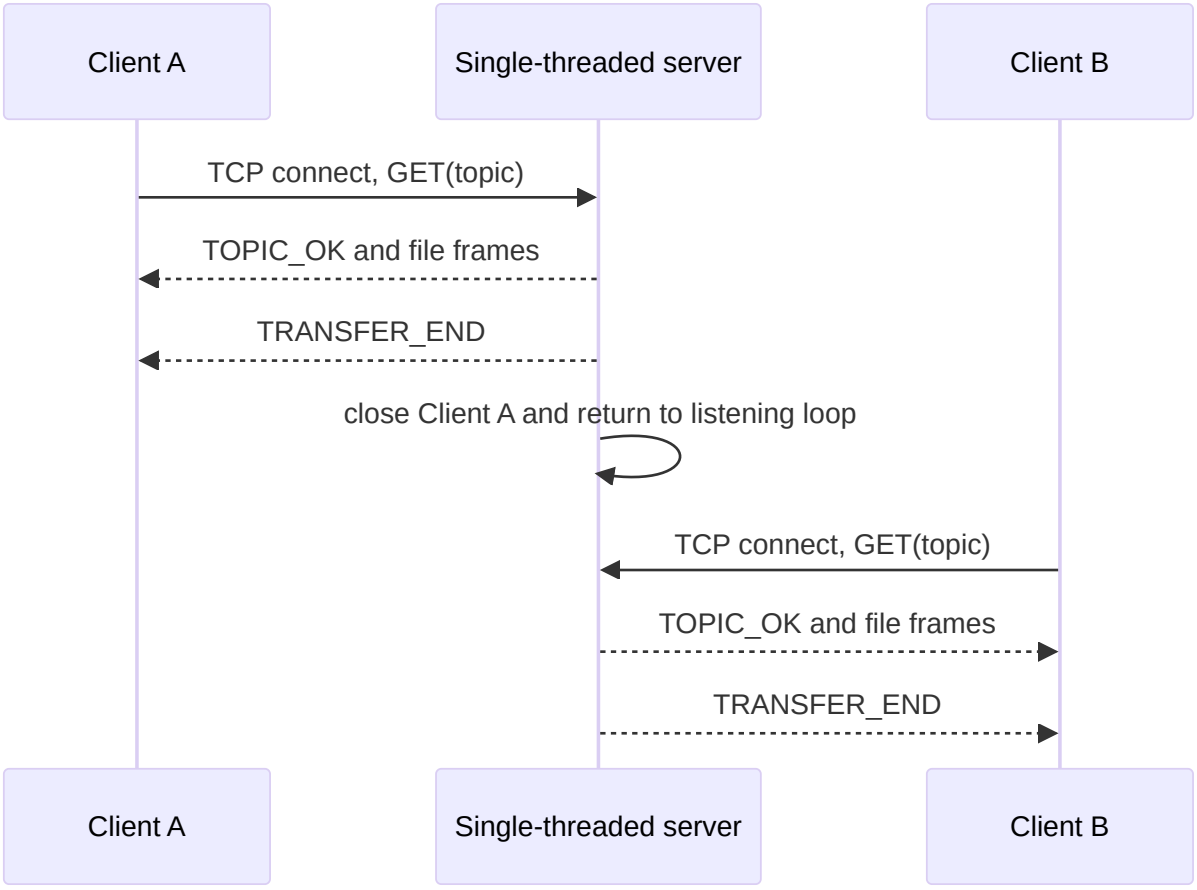
7. Part I Design

7.1 Single-threaded server

The single-threaded server performs the following lifecycle:

1. Parse `<port>` and `<topics_root_dir>` and validate them.
2. Create a TCP socket with `socket()`.
3. Set reusable-address behavior where appropriate, bind with `bind()`, and call `listen()`.
4. Repeatedly call `accept()`.
5. Process one client request in the calling thread: receive `GET`, resolve the topic, enumerate files, send each file, and send `TRANSFER_END`.
6. Close the connected socket and return to `accept()`.
7. On shutdown, close the listening socket and release resources.

A client that arrives while another client is being served waits in the listen backlog. The design is intentionally sequential so that the performance experiment measures the cost of serial service without introducing a hidden concurrency mechanism.

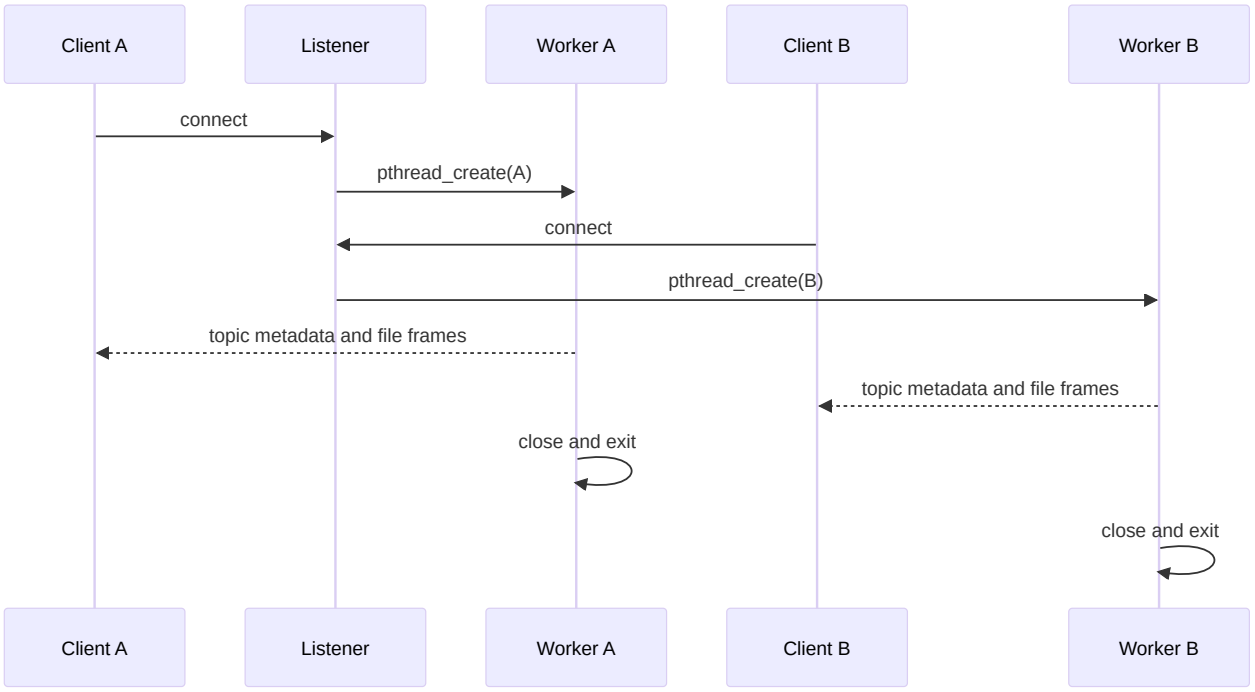


The primary limitation is head-of-line blocking at the server. A slow client delays every later client, so aggregate service time is expected to grow with the number of concurrent requesters.

7.2 Multi-threaded server

The multi-threaded server has one listener thread and one worker per accepted client:

1. The listener parses configuration, creates the listening socket, and enters `accept()`.
2. For each connection, it allocates a small worker argument containing the socket and immutable configuration references.
3. It creates a POSIX thread with `pthread_create()`.
4. The worker receives and validates the request, transfers files, records statistics, closes the connected socket, and exits. Detached threads may be used so that completed workers do not require a join table.
5. The listener continues accepting other clients.



Thread-per-client is selected because it maps directly to the requirement and is easy to inspect during a viva. It also permits blocking file and socket operations without stopping other workers. A process-per-client model would isolate failures but duplicates state and cleanup. A thread pool bounds resource usage but adds a queue and worker lifecycle policy. An event-driven non-blocking design can scale further, but it requires a state machine for every partial I/O event and is unnecessary for the required lab comparison.

---

8. Client Design

The common client workflow is:

1. Parse exactly `./client <server_ip> <server_port> <topic> <output_dir>`.
2. Validate the port, topic component, and output path.
3. Create a TCP socket and call `connect()`.
4. Send the protocol `GET` or `SESSION_START` frame.
5. Receive topic metadata and reject a server error.
6. Create `<output_dir>/<topic>/` using safe directory operations.
7. For every `FILE_BEGIN`, open or create the destination file, seek to the declared start offset, and receive the exact data bytes.
8. Flush and close a file only after its `FILE_END` is validated. A temporary filename or truncation policy prevents a partial file from being mistaken for a complete file.

9. Maintain useful-byte, connection-byte, and checkpoint counters.
10. Detect `recv()` returning zero, a connection error, a malformed frame, or a disk write failure.
11. Apply the part-specific recovery policy.
12. On `TRANSFER_END`, verify all expected files and sizes, log completion, and close the socket.

Part I has no reconnect loop because the transfer is expected to complete or report an error. Case 1 reconnects with `GET` and restarts from file zero. Case 2 reconnects with the session ID and reports the local checkpoint. Case 2 Enhanced coordinates several connections and only declares the topic complete after all work units are committed.

The client must not assume that one `recv()` returns one frame or one file chunk. It reads the fixed header and payload length through helper loops. A timeout or orderly close is treated as an interrupted transfer rather than as successful completion.

---

9. File Transfer Design

9.1 Manifest and file order

The server builds a manifest containing regular files and their sizes. It sorts entries by filename. It sends the count and enough metadata for the client to create deterministic output. The manifest fingerprint is a compact digest or stable metadata string calculated from ordered filenames and sizes. If the directory changes between connections, the server refuses to apply a stale checkpoint rather than silently associating an offset with different content.

9.2 Chunked transfer

A file is transferred as `FILE_BEGIN`, zero or more `FILE_DATA` frames, and `FILE_END`. The server opens the file in binary mode, seeks to `start_offset`, reads at most the configured chunk size into a buffer, and sends the bytes. It repeats until the declared remaining length is zero. Empty files produce `FILE_BEGIN` followed immediately by `FILE_END`.

A large file is not read into one large allocation or passed to one `send()` call. Chunking bounds memory use and makes fault-injection points explicit. It also exposes the implementation to the correct partial-I/O behavior required by TCP systems programming.

9.3 Partial sends and receives

`send_all()` loops until the requested buffer is fully sent, handling short writes and retryable errors. `recv_all()` loops until the requested header or payload length is filled, handling short

reads and orderly close. A return value of zero before the expected number of bytes is a protocol interruption. `EINTR` is retried; other errors are logged and propagated.

Expected helper responsibilities are:

Function	Responsibility
`send_all`	Send exactly a buffer length or return a terminal error.
`recv_all`	Receive exactly a buffer length or report close/error.
`send_frame`	Encode a header and send header plus payload.
`recv_frame`	Decode and validate a header, enforce limits, and receive payload.
`send_file_from_offset`	Send metadata and bounded data chunks from an offset.
`receive_file_from_offset`	Write exact bytes at an offset and verify final size.
`build_manifest`	Enumerate, filter, sort, and stat topic files.

No full source implementation is included in this design document.

---

10. Part II Fault Injection

10.1 Probability meaning

`<failure probability>` is a decimal real number in the inclusive range `[0.0, 1.0]`.

- `0.0` means no injected failure.
- `1.0` means every eligible failure decision interrupts the connection.
- `0.25` means an eligible decision has a 25% probability of interruption.

Values outside the range, NaN-like strings, missing values, and non-numeric text are rejected before the server starts.

10.2 Selected injection policy

The server makes one decision immediately before sending each `FILE_DATA` chunk, after the file metadata has been accepted. This policy ensures that failures occur during active file

transfer, as required, and avoids making metadata behavior the primary experimental variable. The decision is eligible for every chunk of every active transfer. The implementation may also record a decision at the start of a file, but it must use one consistent policy within an experiment.

The random value is generated from a per-process pseudorandom generator seeded once at startup. For reproducible testing, an internal fixed seed may be enabled during a test build or through a non-mandatory configuration mechanism; the required CLI remains unchanged. The server logs the connection/session ID, topic, file index, file offset, configured probability, decision result, and reason for interruption.

When a failure is selected, the worker closes the connected socket without sending a completion frame. An optional `shutdown()` before `close()` makes the interruption explicit, but the client must handle both reset and orderly close. The server marks Case 2's session `INTERRUPTED` after recording the last committed checkpoint. The client detects failure when a required `recv_all()` ends early or returns a socket error.

The random decision does not imply that data already returned by `send()` has reached the client application. Therefore the checkpoint protocol commits progress only through an explicit application-level acknowledgement or a conservative server-side boundary. The selected design uses chunk acknowledgements in Case 2 to make the commitment point precise.

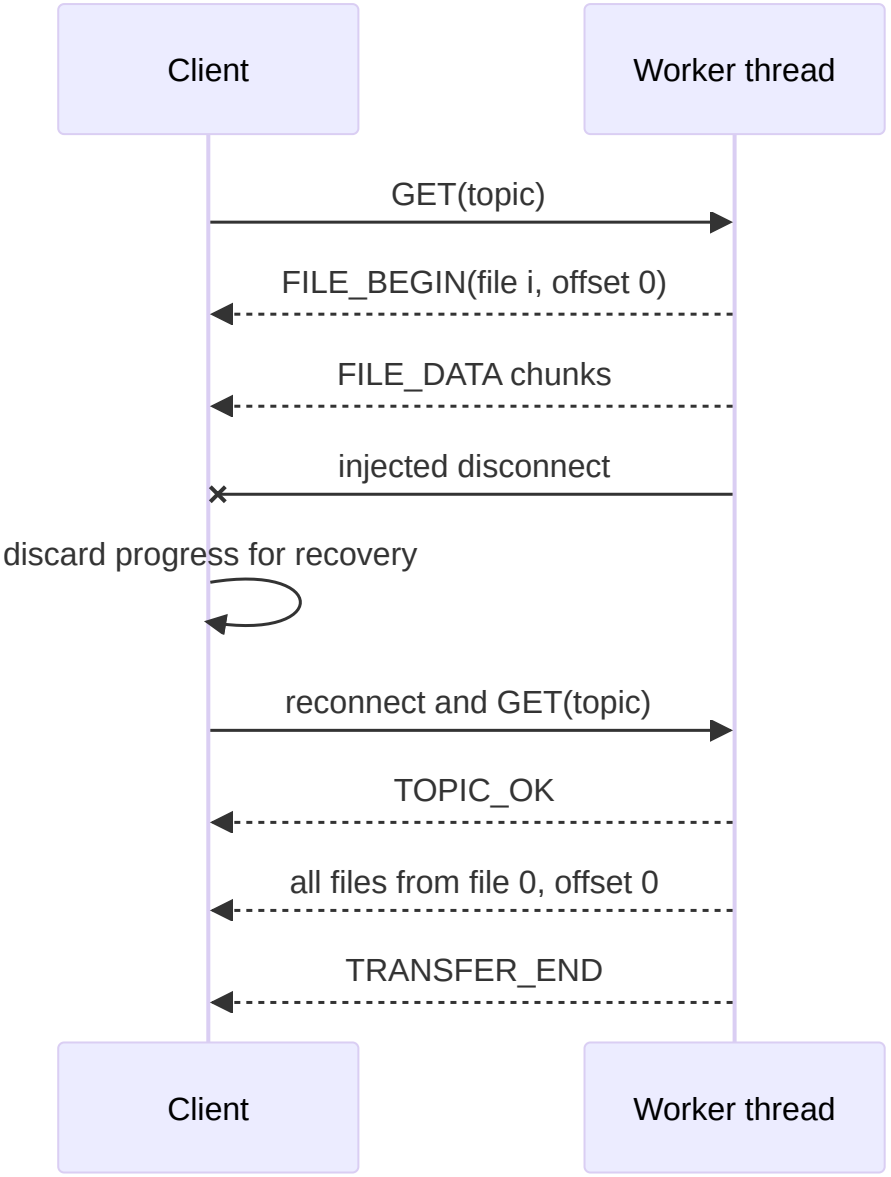
---

11. Part II Case 1: No Session Management

Case 1 uses the multi-threaded server and the ordinary Part I transfer protocol. Fault injection is active, but neither endpoint retains a reusable in-progress transfer record. A local client counter may exist for diagnostics, but it is not sent as a resume checkpoint and does not affect the restart offset.

11.1 Behavior

1. The client sends `GET(topic)`.
2. The server sends files from the first file and offset zero.
3. The server may interrupt before a data chunk.
4. The client detects the incomplete frame or connection close.
5. The client closes the failed socket, waits a bounded retry interval, and reconnects.
6. The client sends the same `GET(topic)`.
7. The server creates a new transfer and sends the entire topic from the beginning.
8. The client replaces or truncates incomplete destination files according to the restart policy and verifies the final result.



11.2 Correctness and overhead

Case 1 is correct if the client does not declare success until every file has been completely rewritten and `TRANSFER_END` has been received. Its main cost is redundant bytes: all bytes sent before the failure may be transmitted again. Planned measurements will distinguish useful final delivery from redundant retransmission.

The client must bound retries or report failure after a configured number of unsuccessful attempts. This prevents a probability of `1.0` from causing an infinite loop during testing.

12. Part II Case 2: Session Management and Resume

12.1 Authoritative state

The server is authoritative for the committed checkpoint. The client stores a local checkpoint so it knows which session to resume, but it cannot advance the server state by assertion alone. The server creates a session record after validating the topic and manifest. It updates the record only at a defined commit point.

The selected commit frequency is **after each acknowledged data chunk**, with a smaller checkpoint record containing file index, offset, and completed-file prefix. This provides bounded retransmission while keeping the protocol understandable. A chunk acknowledgement adds control traffic, but the added traffic is predictable and makes the semantics of “successfully transferred” explicit. A file-level-only checkpoint would be simpler but could repeat an entire large file after a failure.

12.2 Checkpoint protocol

The server sends `FILE_DATA` with a sequence number and byte range. The client writes the bytes, checks that the write completed, and sends `CHUNK_ACK(session_id, file_index, end_offset, sequence_number)`. The server accepts the checkpoint only if it is exactly the next expected range. It then updates the session record under the session mutex. After the final chunk of a file, the client acknowledges `FILE_END`; the server adds that file to `completed_files` and advances the current file.

The checkpoint invariant is:

```
0 ⩽ current_file_offset ⩽ current_file_size
completed_files = all manifest files with index < current_file_index
```

The server does not hold the session mutex while sending the next chunk. It validates and commits a small state transition, then releases the lock.

12.3 Reconnection and resume

After a failure, the client reconnects using the same server address and sends `SESSION_RESUME` with the session ID, topic, manifest fingerprint, and last local checkpoint. The server checks:

1. The session exists and has not expired.
2. The topic and manifest fingerprint match.

3. The requested file index is in range.
4. The filename matches the server manifest entry.
5. The offset is between zero and the current file size.
6. The requested point does not claim completion beyond the server's authoritative checkpoint.

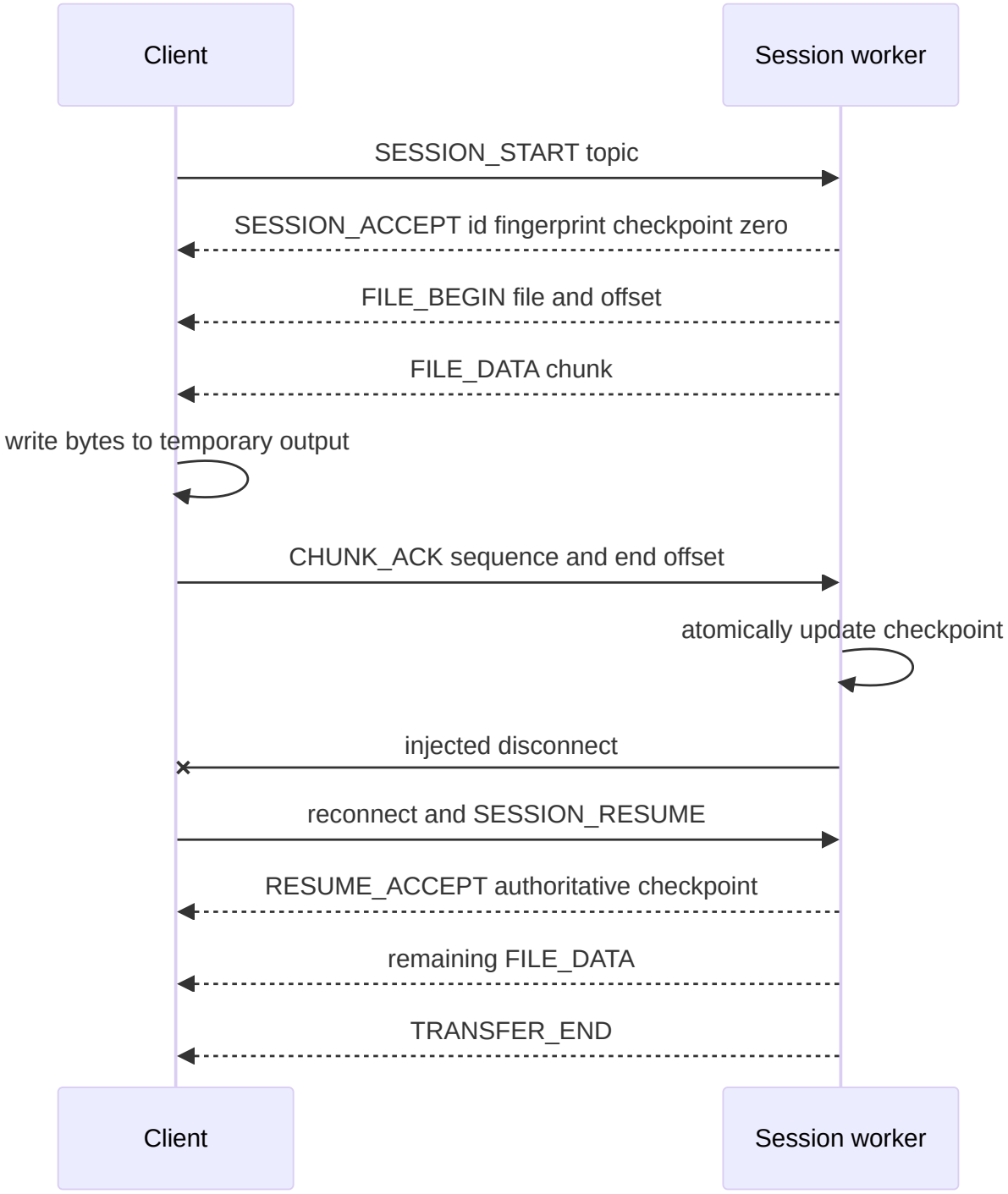
The server responds with `RESUME_ACCEPT` containing the authoritative point. The client truncates or seeks its temporary destination file to that point as necessary, then receives from exactly that point. The server never trusts the client to skip bytes that its own checkpoint does not cover.

12.4 Conflict and edge-case policy

Situation	Server response and client behavior
Client checkpoint is ahead of server	Return the server checkpoint in `RESUME_ACCEPT` with a conflict flag. Client rewinds to the server point. Bytes after it may be overwritten.
Server checkpoint is ahead of client	Return the server checkpoint. Client seeks forward only after confirming its local file contains the required prefix; otherwise it requests retransmission from a conservative earlier point.
Current file no longer exists	Mark the session failed with `FILE_ACCESS`; client reports an unrecoverable transfer error rather than silently substituting another file.
Offset exceeds file size	Return `OFFSET_INVALID`; client resumes at the server's last valid checkpoint or aborts if the manifest changed.
Session expired	Return `SESSION_EXPIRED`; client starts a new session from zero.
Transfer already completed	Return `SESSION_COMPLETE` and completion metadata. Client verifies output and exits successfully without retransmitting.
Duplicate resume request	Return the same authoritative checkpoint if the session is still active; do not create a second logical session.
Manifest changed	Reject the old checkpoint and require a new transfer.

A session expires after a configured inactivity interval. Expiration is checked when a resume arrives and may also be performed by periodic cleanup in the listener or a cleanup worker.

12.5 Case 2 sequence



The client commits a destination file as complete only after the final file acknowledgement and `FILE_END`. This prevents a partially written file from being counted as completed merely because a TCP write returned successfully.

---

13. Part II Case 2 Enhanced

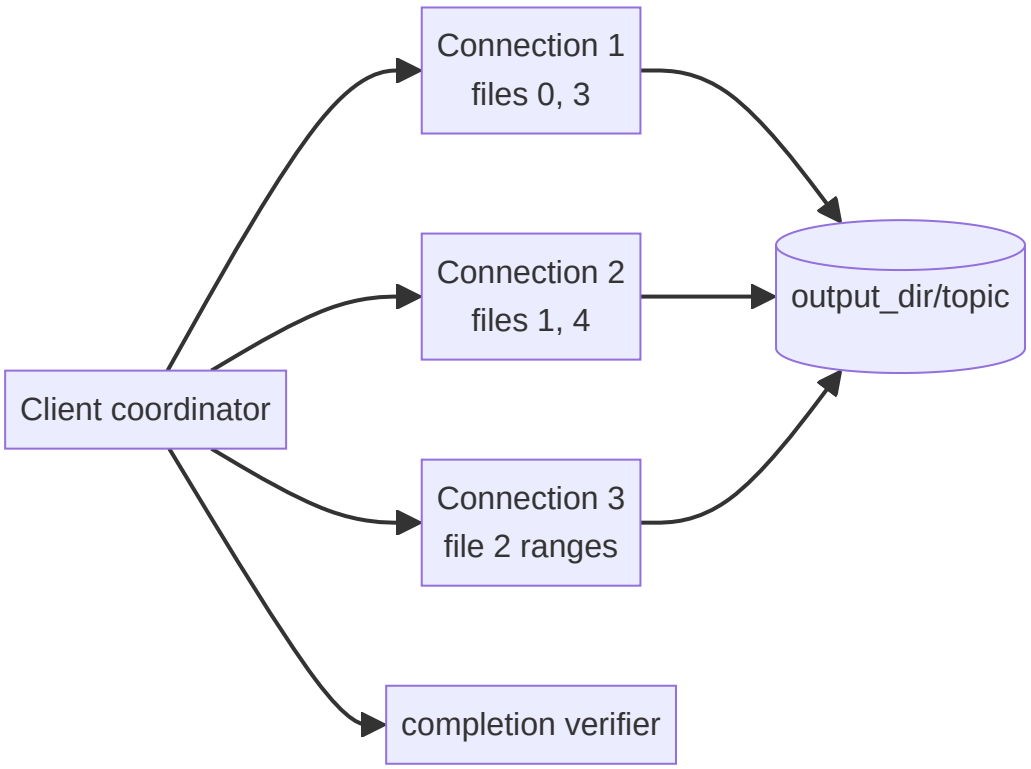
13.1 Selected enhancement

The selected enhancement is **parallel TCP connections with a conservative blocking-socket implementation**. Non-blocking sockets are not required for the enhanced design. The client opens a configurable number of connections, with a default of one so the mandatory invocation works unchanged. The optional value may be supplied as a trailing argument where the test harness permits it.

A coordinator first creates or resumes one logical session. The server creates a work plan from the manifest. Distinct files are assigned to distinct connections when possible. If the topic has fewer files than the requested connection count, only as many workers as useful are created. Large files may be split into non-overlapping byte ranges aligned to chunk boundaries.

13.2 Work split and reassembly

Each work unit contains a unit ID, file index, start offset, and exclusive end offset. The client opens the destination file with random-access writes and places each received chunk at its declared offset. A per-file completion bitmap or interval list records received ranges. The coordinator reports the topic complete only when every work unit has reached `WORK_END` and all file sizes match the manifest.



13.3 Failure recovery

A failed enhanced connection does not invalidate completed units on other connections. The coordinator reports the last acknowledged offset for the failed unit and reconnects using the same session ID. The server reassigns only the missing interval. If the server cannot prove a unit's checkpoint, that unit restarts from its last authoritative boundary, not the whole topic.

The enhanced case must prevent concurrent writes to the same byte range. Separate files are naturally independent. For a split file, ranges are non-overlapping, and the client uses `pwrite()` or an equivalent synchronized seek/write sequence. A mutex is used around shared completion bookkeeping, not around independent network reads.

13.4 Evaluation boundary

The enhanced implementation will be compared with Case 2 using the same dataset, topic, client count, failure settings, and warm-up policy. The design predicts a possible reduction in completion time for workloads where the server, disk, and network can sustain parallelism, but no improvement is claimed before measurement. Parallel connections can also add headers, contention, and scheduling overhead; these costs must be reported if observed.

---

14. Performance Measurement Plan

All values in this section are proposed metrics and planned experiments, not results.

14.1 Timing and counters

Each client records `start_time` immediately before the first request frame and `end_time` after a validated `TRANSFER_END` and output verification. The implementation will use `clock_gettime(CLOCK_MONOTONIC, ...)`, because elapsed time must not be affected by wall-clock adjustments.

The following counters are maintained:

- **Total bytes sent:** bytes passed to the server-side socket `send()` path, optionally including protocol headers. The report will state the selected definition for each table.
- **Useful bytes:** file payload bytes that are part of the final successful topic contents.
- **Redundant bytes:** file payload bytes transmitted but later retransmitted or otherwise not part of the final unique delivery.
- **Completion time:** `end_time - start_time` in seconds.
- **Throughput:** `useful_bytes / completion_time`.

- **Retransmission overhead:** `redundant_bytes / useful_bytes`, with a zero-useful-byte guard.
- **Aggregate file transfer rate for Part I:** total useful file bytes delivered across the measured client group divided by the interval from the first request start until the last client completes.

A byte transmitted before a failure is useful only if it remains part of the final completed output without being overwritten. For Case 1, previously transmitted bytes are therefore counted as redundant when the topic restarts. For Case 2, an acknowledged prefix is useful even if the connection later fails; unacknowledged bytes are conservatively classified according to the final checkpoint rule.

14.2 Planned experiment matrix

Experiment	Independent variables	Measurements
Part I server comparison	Single-threaded vs multi-threaded; clients 1, 2, 4, 8 or available test set	Aggregate rate, completion time, total bytes
Case 1 vs Case 2	Failure probability 0%, 1%, 5%, 10% or selected documented values	Useful bytes, redundant bytes, completion time, throughput, overhead
Enhanced comparison	Case 2 vs Enhanced; connection counts 1, 2, 4 where supported	Completion time, throughput, protocol bytes, recovery time
Fault sensitivity	Fixed client count; several probabilities and repeated trials	Failure count, retries, overhead, successful completion

The final implementation will run on the prescribed group-of-three-computers arrangement where possible: one server node and clients on the other nodes. The dataset, topic, client count, machine placement, compiler flags, and network conditions will be recorded with each run. A warm-up run may be discarded, but this decision must be stated before interpreting results.

Planned outputs include CSV logs, throughput-versus-client-count graphs for Part I, and throughput/overhead/completion-time comparisons for Part II. No graph is included here because no experiment has been performed.

---

15. Error Handling and Edge Cases

Condition	Handling
Invalid topic	Send `TOPIC_ERROR(INVALID_TOPIC)` or `TOPIC_NOT_FOUND`; do not access the filesystem.
Missing topic directory	Return a typed error and close or wait for a new request according to the case.
Empty topic	Send `TOPIC_OK` with zero files and `TRANSFER_END`; client creates the topic directory and verifies empty completion.
Empty file	Send metadata with size zero and an immediate `FILE_END`.
Large file	Stream bounded chunks; never allocate the entire file.
Client disconnect	Worker detects socket error, logs it, and updates session state if enabled.
Server disconnect	Client treats incomplete frame as failure and applies its case-specific recovery.
Partial send/receive	Continue helper loop until complete or terminal error.
Malformed message	Send `MALFORMED_FRAME` when possible, log, and close the connection.
Invalid offset	Reject the checkpoint and return the authoritative valid point or an error.
Invalid probability	Print usage and exit before binding.
Duplicate resume	Return the existing authoritative session state.
Stale session	Return `SESSION_EXPIRED`; restart from zero.
Disk write failure	Stop the transfer, log the path and error, and report `DISK_WRITE`.
Connection timeout	Use a documented socket timeout or bounded retry policy; do not spin indefinitely.

Condition	Handling
Server shutdown	Stop accepting, allow workers a bounded cleanup period, and preserve diagnostic logs.

A retry limit prevents permanent failure injection from creating an infinite client loop. The client reports non-recoverable errors with a non-zero exit status so automated tests can distinguish failure from a completed transfer.

---

16. Concurrency and Synchronization

The listener accesses immutable configuration and a listening socket. Each worker owns its connected socket, request state, file descriptor, and local counters. These resources do not need a shared mutex.

The topic map and manifests may be shared read-only after initialization. If a cache or live rescan is implemented, cache insertion and replacement are protected by a reader/writer policy or a simple mutex. The baseline may build a manifest per request to avoid complicated cache invalidation.

The session table requires a mutex. Session lookup, checkpoint validation, status changes, and expiry must be atomic from the perspective of competing workers. A session record is copied or updated while holding the lock, then the lock is released before file or network operations. This avoids blocking an unrelated client behind a slow disk or socket.

Logging to a shared file or stream is serialized so records are not interleaved. Statistics updates can use a mutex-protected aggregate or per-worker counters merged at completion. Random number generation must either be protected or use per-worker state; the chosen design uses a per-worker generator initialized from a startup seed and worker identity.

Potential races include two reconnecting workers using one session, a cleanup action expiring a session while a resume is validating it, and two enhanced connections claiming the same work unit. The session mutex and work-unit status transitions prevent these races. File content itself is read-only on the server, so concurrent readers are safe under the stated filesystem assumption.

---

17. Module and File Structure

The planned submission layout is:

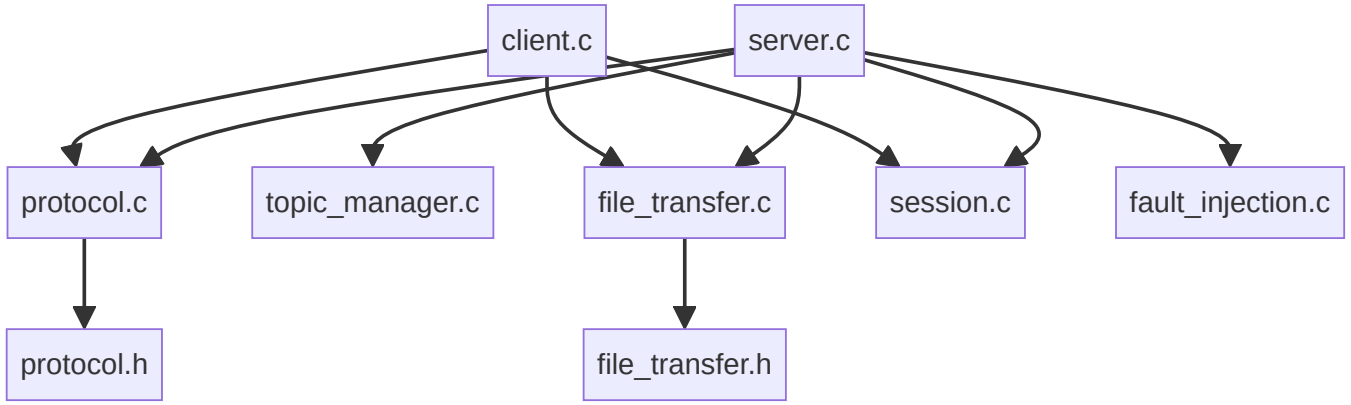
```
Part1/
├── server.c
├── client.c
├── protocol.c
├── protocol.h
├── file_transfer.c
├── file_transfer.h
├── topic_manager.c
├── topic_manager.h
├── logging.c
├── logging.h
└── Makefile

Part2_Case1/
├── server.c
├── client.c
├── protocol.c
├── protocol.h
├── file_transfer.c
├── file_transfer.h
├── topic_manager.c
├── topic_manager.h
├── fault_injection.c
├── fault_injection.h
├── logging.c
├── logging.h
└── Makefile

Part2_Case2/
├── server.c
├── client.c
├── protocol.c
├── protocol.h
├── file_transfer.c
├── file_transfer.h
├── topic_manager.c
├── topic_manager.h
├── session.c
├── session.h
├── fault_injection.c
├── fault_injection.h
├── logging.c
├── logging.h
```

```
└── Makefile

Part2_Case2_Enhanced/
├── server.c
├── client.c
├── protocol.c
├── protocol.h
├── file_transfer.c
├── file_transfer.h
├── topic_manager.c
├── topic_manager.h
├── session.c
├── session.h
├── work_plan.c
├── work_plan.h
├── fault_injection.c
├── fault_injection.h
├── logging.c
├── logging.h
└── Makefile
```



`protocol.*` owns wire constants, integer encoding, framing, and message serialization. `topic_manager.*` validates topic components and produces manifests. `file_transfer.*` performs bounded reads, seeks, and exact byte accounting. `session.*` owns Case 2 state transitions. `fault_injection.*` validates probability and makes eligible decisions. `logging.*` provides synchronized event records and CSV-compatible statistics. The enhanced `work_plan.*` maps files or ranges to connection units.

The four directories may share copied modules or a carefully documented common module during development, but each final directory must build independently and produce the required executables.

18. Planned Key Functions

Function	Module	Purpose	Used in
`parse_server_args`	configuration	Parse and validate server arguments.	All
`parse_client_args`	client	Parse the exact client CLI.	All
`create_server_socket`	server	Create, bind, and listen.	All
`accept_client`	server	Accept one connection.	All
`handle_client`	server	Dispatch a connected client transaction.	All
`find_topic`	topic manager	Resolve a safe topic name.	All
`build_manifest`	topic manager	Enumerate and sort regular files.	All
`send_frame`	protocol	Serialize and send one frame.	All
`receive_frame`	protocol	Receive and validate one frame.	All
`send_all` / `recv_all`	protocol	Correctly handle partial I/O.	All
`send_file_from_offset`	file transfer	Stream a file range.	All
`receive_file_from_offset`	file transfer	Write and verify an exact range.	All
`inject_failure_if_eligible`	fault injection	Make and log a probabilistic decision.	Part II

Function	Module	Purpose	Used in
`create_session`	session	Allocate a new logical transfer.	Case 2
`validate_resume`	session	Compare client state with server authority.	Case 2
`commit_checkpoint`	session	Atomically update a chunk boundary.	Case 2
`expire_sessions`	session	Remove or mark stale sessions.	Case 2
`build_work_plan`	work plan	Assign independent files/ranges.	Enhanced
`record_statistics`	logging	Persist counters and timings.	All
`verify_completion`	client	Check file count and sizes before success.	All

These are interfaces and responsibilities, not complete implementations.

---

19. Build and Execution Design

Each case will be built from its own directory:

```
cd Part1 && make
cd ../Part2_Case1 && make
cd ../Part2_Case2 && make
cd ../Part2_Case2_Enhanced && make
```

The resulting directory must contain directly runnable executables named `server` and `client`.

Required invocations are unchanged:

```
# Part I
./server <port> <topics_root_dir>
./client <server_ip> <server_port> <topic> <output_dir>

# Part II, including Case 1, Case 2, and Enhanced
./server <port> <topics_root_dir> <failure probability>
./client <server_ip> <server_port> <topic> <output_dir>
```

The enhanced client may accept an optional trailing connection-count argument if a default exists and the required four-argument invocation remains valid. The server must not require interactive input. All paths and network values used for a run come from arguments or documented defaults. This allows an automated grader to substitute its own dataset and port.

A planned test harness will start the server in the background, wait for the listening port, run one or more clients, compare output files with the source manifest, and terminate the server. It will also run invalid-argument tests and fault-injection tests with bounded timeouts.

---

20. Design Alternatives and Decisions

Problem	Alternatives	Selected design	Reason
Client concurrency	Process per client; thread pool; thread per client	Thread per client	Directly satisfies the lab requirement and keeps blocking I/O simple.
Socket mode	Blocking; non-blocking with `poll()` / `epoll()`	Blocking baseline	Fewer states and clearer mapping between one worker and one transfer.
Checkpoint authority	Client; server; both with server authority	Both, server authoritative	Client reconnects with local knowledge while the server prevents false advancement.
Checkpoint frequency	File; periodic bytes; every chunk	Every acknowledged chunk	Bounds retransmission and gives an explicit commit point.
Framing	Newline text; delimiter escaping; fixed header plus length	Fixed header plus explicit length	Works for arbitrary binary data and partial TCP reads.
File scheduling	Sequential files; parallel files; byte-range striping	Sequential baseline; parallel units in Enhanced	Baseline is easy to verify; enhanced isolates concurrency.
Session ID	Topic only; random opaque ID; deterministic composite ID	Random opaque ID plus topic/fingerprint validation	Avoids collisions and keeps identity independent of path strings.
Fault timing	Once per connection; once per file; once per chunk	Before each data chunk	Provides repeated eligible decisions during active

Problem	Alternatives	Selected design	Reason
			transfer and is easy to log.
Session persistence	In-memory; durable file/database	In-memory with TTL	Sufficient for a single server process and avoids new storage dependencies.
Output handling	Write final files directly; temporary files	Temporary/seekable output then final verification	Prevents partial files from being treated as completed.

The selected designs are intentionally modest. They provide the behaviors needed for the assignment without introducing an application framework or a durable database that would distract from socket programming.

---

21. Robustness and Safety Considerations

The server must prevent path traversal. A topic request is accepted only if it is a valid single directory component and does not contain `/`, `\\`, `..`, or embedded NUL bytes. The server constructs the candidate path below the canonical root and verifies that it is a directory. A filename received from the server manifest is generated from a directory entry, but it is still validated before being placed under the client output topic directory.

The server does not accept an arbitrary client-supplied filesystem path. It restricts access to the configured root and transfers only regular files. It validates manifest indexes, offsets, file sizes, frame lengths, and session IDs. Protocol limits protect against unbounded allocations. The client also validates that a server filename cannot escape `<output_dir>/<topic>/`.

The design does not attempt to provide encryption, authentication, or Internet-scale authorization. The service is intended for the controlled lab network. It nevertheless handles malformed requests, invalid arguments, resource errors, and abrupt disconnects explicitly.

---

22. Logging and Additional Features

The following small features are planned because they aid implementation and evaluation without changing the required protocol:

Feature	Purpose	Why useful	Implementation impact
Structured transfer log	Record connection/session, topic, event, offset, and result.	Makes injected failures and recovery observable.	A small synchronized logger module.
CSV-compatible statistics	Store timing and byte counters.	Supports later graphs without manual transcription.	One record at completion and optional event records.
Clear error codes	Separate invalid input, missing data, and protocol failure.	Helps automated tests and viva diagnosis.	Shared constants in `protocol.h`.
Graceful shutdown	Stop accepting and close active resources in a bounded way.	Prevents confusing test artifacts.	Signal flag and listener cleanup.
Session cleanup	Mark inactive sessions expired.	Prevents unbounded memory growth during repeated fault tests.	TTL check on resume and cleanup pass.
Configurable log level	Suppress or enable diagnostic detail.	Allows clean automated output and detailed debugging.	Optional environment/configuration setting, not a required CLI argument.

No feature is allowed to change the mandatory commands or make the test harness depend on interactive behavior.

---

23. End-to-End Workflows

23.1 Part I single-threaded

1. Start `server` with a port and root directory.
2. Validate arguments and prepare the listening socket.
3. Accept Client A.
4. Receive `GET(topic)` and build a manifest.
5. Send metadata and all file frames from offset zero.
6. Send `TRANSFER_END`, close Client A, and update statistics.
7. Accept Client B and repeat.
8. Compare completion time and aggregate rate for planned client-count experiments.

23.2 Part I multi-threaded

1. Start the same service in the multi-threaded directory.
2. Accept each client and create a worker.
3. Each worker resolves its topic and sends files independently.
4. The listener continues accepting new clients.
5. Workers close their sockets and record per-client completion.
6. The experiment aggregates bytes and the interval covering all clients.

23.3 Case 1

1. Start the multi-threaded Part II server with a probability.
2. Client sends `GET(topic)`.
3. Worker streams files and may close the socket before a data chunk.
4. Client identifies incomplete transfer and discards recovery progress.
5. Client reconnects and repeats `GET(topic)`.
6. Server restarts from file zero.
7. Client retries up to the bounded limit, then verifies the complete output.

23.4 Case 2

1. Client sends `SESSION_START(topic)` and receives a session ID.
2. Server sends file metadata and data chunks.
3. Client writes each chunk and acknowledges it.
4. Server commits each acknowledged checkpoint.
5. A fault closes the connection.
6. Client reconnects with `SESSION_RESUME` and its local checkpoint.
7. Server validates the session and returns the authoritative checkpoint.
8. Client rewinds or advances its temporary output to the returned point.
9. Server sends the remaining ranges and `TRANSFER_END`.
10. Client verifies files and logs useful/redundant bytes.

23.5 Case 2 Enhanced

1. Client starts a logical session and negotiates a connection count, defaulting to one.
2. Server returns a work plan of files or non-overlapping ranges.
3. Client opens several TCP connections and assigns one work unit to each.
4. Workers receive and place data independently while acknowledging unit progress.
5. A failed connection reconnects and requests only its missing unit range.
6. The coordinator waits until all units and final file sizes are complete.
7. Client records total time, useful bytes, redundant bytes, and connection overhead.

---

24. Verification and Automated-Test Compatibility

Verification will be based on observable behavior rather than undocumented assumptions. For each case, tests will use a temporary root directory with known topic directories and files, including empty files and binary data. The client output will be compared byte-for-byte with the source topic files.

Protocol tests will exercise short reads and writes where possible, malformed frame lengths, invalid topics, invalid ports, and early connection closure. Fault tests will use probability zero to establish a no-failure baseline and probability one to confirm that the retry limit and recovery path terminate predictably. Case 1 must restart from the beginning; Case 2 must retain a server-authoritative checkpoint and avoid retransmitting acknowledged prefixes.

The automated interface constraints are treated as acceptance conditions:

- `make` in every required directory produces `server` and `client`.
- Server and client accept exactly the required mandatory arguments.
- No source file embeds a dataset-specific absolute path, fixed IP, or fixed port.
- The output appears under `<output_dir>/<topic>/`.
- A non-zero exit status is used for unrecoverable errors.
- The source tree can be run against a grader-supplied root directory.

Performance validation will use the same topic and client population for comparable variants. Results will be stored as raw logs before graphs are generated. The design document itself contains no experimental conclusion.

---

25. Requirements Traceability Matrix

Assignment requirement	Design section	Planned implementation component	Verification method
Topic maps to directory	5	`topic_manager.*`	Request every test topic and inspect manifest.
TCP socket communication	6–9	`server.c`, `client.c`, `protocol.*`	End-to-end transfer over TCP.
Part I single-threaded server	7.1	`Part1/server.c`	Multiple clients complete sequentially.
Part I multi-threaded server	7.2	`Part1/server.c`	Concurrent clients complete without cross-talk.
Client requests `GET <topic>`	6.3, 8	`client.c`, `protocol.*`	Capture/log request and output files.
All topic files transferred	9	`file_transfer.*`	Byte-for-byte output comparison.
Fault probability CLI	10	`fault_injection.*`, Part II `server.c`	Test 0.0, 1.0, and invalid values.
Active connection interruption	10	`inject_failure_if_eligible`	Observe logged close during data transfer.
Case 1 restarts from beginning	11	Case 1 client/server	Compare repeated bytes and final correctness.
Case 2 retains session state	12	`session.*`	Inspect checkpoint logs and resume behavior.

Assignment requirement	Design section	Planned implementation component	Verification method
Resume from validated offset	12	`validate_resume`, `commit_checkpoint`	Inject failure after an acknowledged chunk.
Checkpoint conflict handling	12.4	`session.*`	Send ahead/behind/invalid offsets.
Enhanced performance mechanism	13	`work_plan.*`, enhanced client/server	Compare connection-count variants.
Partial send/receive	6.1, 9.3	`send_all`, `recv_all`	Stress with varied chunking and loopback tests.
Performance timing	14	`logging.*`, client statistics	Inspect monotonic timestamps and CSV fields.
Throughput vs client count	14	test harness and `Results/`	Planned multi-client matrix.
Useful/redundant bytes and overhead	14	statistics counters	Reconcile counters with final files.
Required CLI and no hard-coded paths	19, 24	all `server` / `client` entry points	Run against alternate root and port.
Required source layout	17, 19	four case directories and Makefiles	Build each directory independently.
Error handling	15	protocol, topic, file, session modules	Negative and disconnect tests.
Synchronization	16	session, logging, statistics modules	Concurrent resume and log-integrity tests.

Assignment requirement	Design section	Planned implementation component	Verification method
Additional useful features	22	logging, cleanup, graceful shutdown	Inspect logs and bounded shutdown.
Design alternatives and decisions	20	design baseline	Viva explanation and implementation review.
No fabricated results	14, 26	report process	Add results only after experiments.

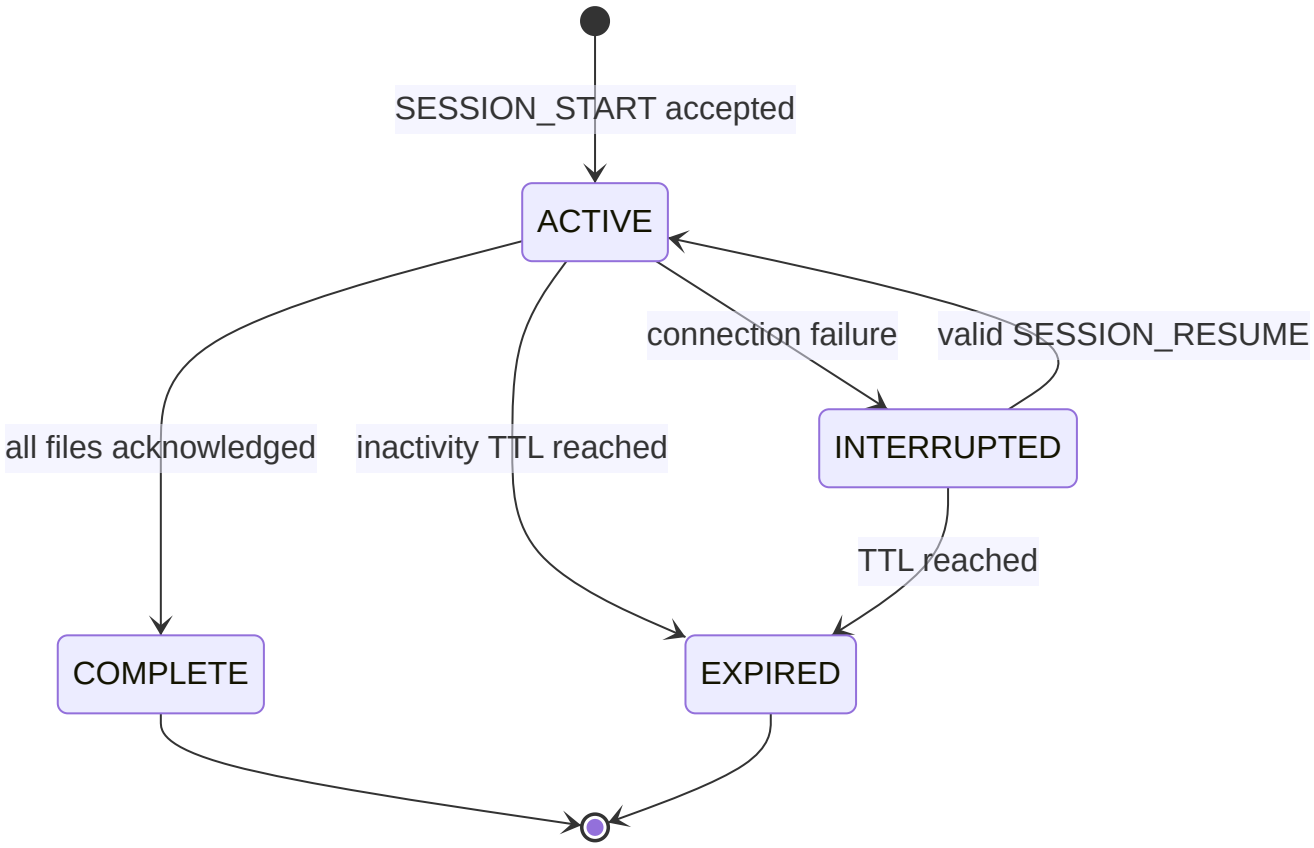
26. Expected Design Outcome

The selected baseline is a blocking TCP service with a single-threaded Part I variant and a thread-per-client multi-threaded variant. Both use the same length-framed, binary-safe application protocol and deterministic topic manifests. Part I establishes the correctness and performance baseline.

Part II extends the multi-threaded server with configurable fault injection. Case 1 demonstrates the cost of losing all progress: a reconnect repeats the entire topic. Case 2 introduces a server-authoritative checkpoint, acknowledgements at chunk boundaries, and a resume protocol that validates session identity, manifest, file, and offset. Case 2 Enhanced adds parallel TCP work units while retaining a one-connection default and independent recovery of incomplete units.

All variants preserve the required command-line interface and avoid hard-coded dataset paths, ports, or addresses. The planned timing and byte counters support later comparison of throughput, completion time, useful bytes, redundant bytes, and retransmission overhead. The final implementation and experiments will determine whether the enhanced design improves measured performance; this design document makes no such claim in advance.

Appendix A — Session state machine



The state machine prevents a completed session from being silently reused and gives the resume path a clear terminal response for expired sessions.

Appendix B — Conceptual protocol frame

```
+--------+---------+------+------+----------+----------------+
| magic  | version | type |flags | reserved | payload_len    |
| 2 B    | 1 B     | 1 B  | 2 B  | 2 B      | 8 B            |
+--------+---------+------+------+----------+----------------+
| payload_len bytes of typed payload                         |
+------------------------------------------------------------+
```

The explicit payload length is the boundary that allows the receiver to distinguish control metadata from arbitrary binary file bytes.